

MODULE 09 · PART 2 - WINDOW FUNCTIONS

Distributions and Percentiles

Where a value sits in a spread, rather than where it sits in a queue. And the median, which is harder than it should be.

PART 2 of 8 MODULE 9 of 26 RUNS about 30 min

Why proportions travel and ranks do not

- ◆ **Third of four hundred and twelve and third of nine are not comparable.** A rank needs its denominator quoted alongside it.
- ◆ **A proportion carries its denominator inside it**, so it means the same thing in a region of forty stores and one of eighty.
- ◆ **Which is why every cross-group comparison ends up here**, whether or not anybody planned it.
- ◆ **And why an unqualified rank in a report is nearly always a mistake.** Third of how many is the question nobody asked.

THE TAKEAWAY

Kestrel's regions hold between forty and eighty stores. A league table of ranks across regions is comparing two different scales and calling it one.

A rank is a **position**. A percentile is a **proportion**. They answer different questions about the same row.

The family, in one table

FUNCTION	WHAT IT RETURNS
<code>PERCENT_RANK()</code>	Rank as a fraction. Runs 0 to 1, and the first row is exactly 0.
<code>CUME_DIST()</code>	Share of rows at this value or below. Never 0, and the last row is 1.
<code>NTILE(n)</code>	A bucket number, 1 to n, by row count rather than by value.
<code>PERCENTILE_CONT(p)</code>	The value at proportion p, interpolated. May never have been observed.
<code>PERCENTILE_DISC(p)</code>	The value at proportion p, taken from the data. Always a real observation.

THE TAKEAWAY

The first three take a row and return a proportion. The last two take a proportion and return a value. That inversion is the shape of the module.

The two rank fractions

```
1 SELECT store_key, daily_sales,  
2         RANK() OVER w AS rnk,  
3         PERCENT_RANK() OVER w AS pct_rank,  
4         CUME_DIST() OVER w AS cume_dist  
5 FROM agg_store_day  
6 WHERE sale_date = DATE '2026-08-31'  
7 WINDOW w AS (PARTITION BY region_key ORDER BY daily_sales);
```

7 No frame anywhere. Both are ranking-family functions, so they always see the whole partition.

THE TAKEAWAY

Both are derived from RANK, so both inherit its tie behaviour and neither takes a frame. Everything they do follows from the rank underneath.

Eight rows, worked through

VALUE	RANK PERCENT_RANK CUME_DIST
10	1 0.000 0.125
20	2 0.143 0.375
20	2 0.143 0.375
30	4 0.429 0.500
40	5 0.571 0.875
40	5 0.571 0.875
40	5 0.571 0.875
50	8 1.000 1.000

THE TAKEAWAY

PERCENT_RANK is $(\text{rank} - 1) / (n - 1)$. CUME_DIST is the count of rows at this value or lower, over n. Two formulas, and every difference follows.

PERCENT_RANK or CUME_DIST

PERCENT_RANK

How far along the queue

- First row is always exactly 0
- Last row is always exactly 1
- Denominator is n minus one
- Excludes the current row

CUME_DIST

What share is at or below

- First row is $1/n$, never 0
- Last row is always 1
- Denominator is n
- Includes the current row

THE TIEBREAKER

If a top performer should read as 1.00 rather than 0.00, you want CUME_DIST. If you want a symmetric 0 to 1 scale, you want PERCENT_RANK.

Why the first row is exactly zero

- ◆ **PERCENT_RANK counts rows strictly better, over the rows other than this one.** The best row has none better, so it is 0.
- ◆ **That is a scale from 0 to 1, not a percentage of anything.** A store at 0.00 is the best in its region, and it reads as a failure.
- ◆ **And in a partition of one row it is undefined,** dividing by zero. Most engines return 0 there by definition.
- ◆ **CUME_DIST has neither problem,** because its denominator is n and the current row is always counted.

THE TAKEAWAY

A metric that reads as zero for the best performer will be misread by somebody. That is not a defect in the function, and it is a defect in the report.

A store performance pack shows each store's position within its region using PERCENT_RANK over daily sales, ascending. Regions hold between forty and eighty stores.

A regional manager escalates that their flagship store, the highest-selling site in the region, shows a score of 0.00 in every week of the pack. They read it as the store scoring nothing. The figure is correct in every week.

What is happening?

- A

 PERCENT_RANK is $(\text{rank} - 1) / (n - 1)$, so the first row is exactly 0 by construction. CUME_DIST would read 1.00 for that store.
- B

 The ordering is ascending when it should be descending, so the ranking runs the wrong way round.
- C

 The partition is wrong. Regions should be partitioned by trading week as well as by region.
- D

 Nulls in daily_sales sort first on Redshift, taking the top positions and pushing real stores down.

◆ PAUSE AND DECIDE

What is happening?

- A

The formula starts at zero

The best row has no rows strictly better than it, so the numerator is zero. CUME_DIST gives that store 1.00 and nobody escalates.
- B

Wrong sort direction

Ascending is deliberate here and the manager's store is the last row, not the first. Reversing it makes the worst store read 0.00 instead.
- C

Partition by week too

The pack is already weekly, so this changes nothing. It is the kind of answer that sounds structural and moves no numbers.
- D

Nulls sorting first

Nulls are greatest on Redshift, so they sort last ascending. And the scenario states the figure is correct, which rules out a data issue.

THE SIGNAL

"The figure is correct in every week." A number that is right, escalated by the person it describes, is a presentation problem. The fix is the other function.

Where these two beat NTILE

- ◆ **NTILE divides by row count and splits ties across buckets.** Module four's scenario three, and the fairness problem in it.
- ◆ **PERCENT_RANK and CUME_DIST give tied rows the same value,** always, because they are built on RANK.
- ◆ **So a threshold on CUME_DIST cannot separate two identical customers,** which is what the loyalty team needed.
- ◆ **NTILE is still right when you need equal-sized groups,** and that is a real requirement. It is just not most of them.

THE TAKEAWAY

If a boundary decides who gets something, use a value-based cut. If you need exactly ten groups of equal size, use NTILE and accept the arbitrary edge.

Top decile, three ways

```
1 -- by count of rows
2 QUALIFY NTILE(10) OVER (ORDER BY spend DESC) = 1
3
4 -- by proportion of rows, ties kept together
5 QUALIFY CUME_DIST() OVER (ORDER BY spend DESC) <= 0.10
6
7 -- by value, against a computed threshold
8 QUALIFY spend >= PERCENTILE_CONT(0.90)
9     WITHIN GROUP (ORDER BY spend) OVER ()
```

- 5 Cume dist keeps every customer tied at the boundary, so the set may be slightly larger than a tenth.
- 8 A value threshold. Two customers on identical spend are always treated identically, by construction.

THE TAKEAWAY

Three answers to 'the top ten percent', and they return different sets. Only the third is defensible when the membership decides who gets something.

'The top ten percent' is three requests

- ◆ **Ten percent of the customers**, which is a count, and gives you a fixed-size list.
- ◆ **Customers above the 90th percentile of spend**, which is a value, and gives you a variable-size list.
- ◆ **Customers who make up ten percent of the revenue**, which is a cumulative share, and is module eight's Pareto.
- ◆ **All three are said the same way**. The third is usually what a commercial stakeholder means and the first is what gets built.

THE TAKEAWAY

Ask which denominator: customers, or spend. The phrase 'top ten percent' does not contain the answer and the query silently supplies one.

Reading a distribution measure out loud

WRITTEN	SAID
<code>PERCENT_RANK() OVER (ORDER BY x)</code>	how far along the queue this row sits, 0 to 1
<code>CUME_DIST() OVER (ORDER BY x)</code>	the share of rows at this value or lower
<code>NTILE(10) OVER (ORDER BY x)</code>	which tenth of the rows, by count, this one falls in
<code>PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY x)</code>	the value at the ninetieth percentile, interpolated
<code>PERCENTILE_DISC(0.9) WITHIN GROUP (ORDER BY x)</code>	the smallest observed value with at least 90% at or below

THE TAKEAWAY

Rows one and two sound similar and differ at both ends. Rows four and five sound identical and differ whenever the count is even.

Small partitions break these measures

- ◆ **PERCENT_RANK in a partition of one divides by zero.** Engines return 0, so a lone store reads as the worst possible score.
- ◆ **CUME_DIST in a partition of one returns 1,** so the same store reads as the best. Neither is informative.
- ◆ **A percentile of three values is barely a percentile.** The ninetieth and the maximum are the same number.
- ◆ **Carry the partition size.** COUNT(*) OVER the same partition, so a reader can discount a figure from a group of four.

THE TAKEAWAY

Every measure in this module is meaningless below some group size, and none of them says so. The count alongside is the only warning anybody gets.

Mean and median on the same rows

```
1 SELECT store_key,  
2        AVG(basket_value)                AS mean_basket,  
3        PERCENTILE_CONT(0.5) WITHIN GROUP (  
4        ORDER BY basket_value)          AS median_basket,  
5        PERCENTILE_CONT(0.9) WITHIN GROUP (  
6        ORDER BY basket_value)          AS p90_basket,  
7        MAX(basket_value)                AS max_basket,  
8        COUNT(*)                        AS baskets  
9 FROM vw_basket  
10 GROUP BY store_key;
```

4 On right-skewed data the mean sits above the median, usually well above. The gap is the skew.

THE TAKEAWAY

Publish both and the shape of the distribution becomes visible. A mean well above a median is the signal that the mean is describing a minority of baskets.

So far, a row goes in and a proportion comes out. **Percentiles run the other way.**

WITHIN GROUP, a clause you have not met

```
1 SELECT region_key,  
2        PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY basket_value)  
3        AS median_basket,  
4        PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY basket_value)  
5        AS median_observed,  
6        PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY basket_value)  
7        AS p90_basket  
8 FROM vw_basket  
9 GROUP BY region_key;
```

- 2 The proportion is the argument. The column being measured lives in WITHIN GROUP, not in the parentheses.
- 9 An ordinary GROUP BY. These are aggregates, so they collapse rows like any other aggregate does.

THE TAKEAWAY

The column you are measuring is not where you expect it. That is the single most common syntax error with these functions, on every engine.

Continuous or discrete, on four values

ORDERED VALUES: 10, 20, 30, 40	RESULT
<code>PERCENTILE_CONT(0.5)</code>	25. The midpoint of the two middle values.
<code>PERCENTILE_DISC(0.5)</code>	20. The first value whose CUME_DIST reaches 0.5.
<code>PERCENTILE_CONT(0.25)</code>	17.5. Interpolated between 10 and 20.
<code>PERCENTILE_DISC(0.25)</code>	10. An actual observation.
Does 25 appear in the data?	No. It is computed, and it is still the correct median.

THE TAKEAWAY

CONT can return a number nobody ever recorded. DISC always returns something that happened. Neither is wrong, and only one of them is auditable.

Interpolation invents a value

- ◆ **PERCENTILE_CONT** returns a number that may never have been observed. That is its definition, not a flaw.
- ◆ **Which makes it untraceable.** You cannot go and look at the basket that had that value, because there was not one.
- ◆ **And it can be nonsensical for discrete things.** A median basket of 2.5 items, or a median of a categorical code.
- ◆ **PERCENTILE_DISC** is the audit-safe choice, because every value it returns is a row somebody can go and find.

THE TAKEAWAY

Ask whether anybody will need to look up the row behind the number. If they will, the continuous version cannot give them one.

Percentiles depend on position, not value

- ◆ **The median of 1, 2, 3, 4, 1000 is 3.** The extreme value moves the mean enormously and moves the median not at all.
- ◆ **That is the whole reason to use one.** Kestrel's basket values are right-skewed, so the mean sits above most baskets.
- ◆ **And it is why medians cannot be combined.** Knowing each store's median tells you nothing about the group's.
- ◆ **Means are additive and medians are not.** That single property decides how a percentile has to be computed.

THE TAKEAWAY

A mean can be rebuilt from summed numerators and summed denominators. A median cannot be rebuilt from anything except the original rows.

Kestrel reports median basket value per store, computed with `PERCENTILE_CONT(0.5)`. One store's figure is \$18.47. An auditor asks to see the receipt at the median, as a spot check on the calculation.

The analyst searches the day's baskets for that store and finds nothing at \$18.47. The nearest values are \$18.44 and \$18.50. The store took an even number of baskets that day.

What should the analyst say?

- ☐ A It is a rounding artefact. Report the nearest real basket, at \$18.50, and adjust the rounding in the pipeline.
- ☐ B The median is being computed over the wrong grain, so it does not correspond to any single basket.
- ☐ C Medians are not traceable in principle. Use `AVG`, which can always be reconciled back to the underlying rows.
- ☐ D `PERCENTILE_CONT` interpolates between the two middle values, so on an even count it returns a value never observed.

◆ PAUSE AND DECIDE

What should the analyst say?

A A rounding artefact

18.47 is the exact midpoint of 18.44 and 18.50. Nothing was rounded, and calling it rounding would make the next audit worse.

C Use AVG instead

Traceable and a different statistic. Kestrel's basket values are right-skewed, which is why the median was chosen over the mean.

B Wrong grain

A real hazard with basket measures and not this one. The grain is baskets, and the value is exactly between the two middle baskets.

D Interpolation, by definition

An even count has no single middle row, so CONT averages the two. PERCENTILE_DISC would return 18.44, which the auditor can look up.

THE SIGNAL

"The nearest values are 18.44 and 18.50." Their midpoint is 18.47 exactly. The arithmetic identifies the cause before anybody opens the query.

Mean, median, mode

- ◆ **The mean uses every value.** One extreme observation moves it, which is either sensitivity or fragility depending on the question.
- ◆ **The median uses only the middle position.** Robust, untraceable in its continuous form, and impossible to roll up.
- ◆ **The mode is the most frequent value.** The only one of the three that is meaningful for categories.
- ◆ **For revenue you want the mean. For typical you want the median.** Totals divide into means, and never into medians.

THE TAKEAWAY

The mean is the only one that reconciles to a total. That is why finance uses it and why it keeps being the wrong answer for 'what does a typical shopper spend'.

MODE, the forgotten one

```
1 SELECT store_key,  
2         MODE() WITHIN GROUP (ORDER BY tender_key) AS usual_tender,  
3         MODE() WITHIN GROUP (ORDER BY day_of_week) AS busiest_dow  
4 FROM vw_basket  
5 GROUP BY store_key;
```

- 2 Same ordered-set syntax as the percentiles. Postgres, Redshift, Snowflake and Oracle all have it.
- 3 With ties the standard returns one of them, and which one is engine-specific. Check before relying on it.

THE TAKEAWAY

The only one of the three summaries that works on a category. A mean tender type is meaningless and a median tender type is nearly as bad.

Percentiles and nulls

- ◆ **Nulls are ignored in the calculation.** Snowflake's documentation says so directly, and every engine behaves the same way.
- ◆ **So the denominator is the non-null count**, exactly as with AVG, and exactly as invisibly.
- ◆ **Which can shift a median a long way** if the nulls are not missing at random. Unrecorded baskets are rarely a random sample.
- ◆ **Publish the non-null count alongside**, so the reader knows what proportion of the population the figure describes.

THE TAKEAWAY

Module two's denominator problem, arriving in a function where the denominator is even harder to see, because there is no division written anywhere.

The canonical use: p90 and p99

```
1 SELECT screen,  
2     COUNT(*)           AS events,  
3     AVG(load_ms)       AS mean_ms,  
4     PERCENTILE_CONT(0.50) WITHIN GROUP (  
5         ORDER BY load_ms) AS p50_ms,  
6     PERCENTILE_CONT(0.90) WITHIN GROUP (  
7         ORDER BY load_ms) AS p90_ms,  
8     PERCENTILE_CONT(0.99) WITHIN GROUP (  
9         ORDER BY load_ms) AS p99_ms  
10 FROM fact_app_event  
11 GROUP BY screen;
```

9 The ninety ninth percentile of 892 million events is roughly nine million of them. Not an edge case.

THE TAKEAWAY

The mean latency is the experience of nobody in particular. The p99 is the experience of nine million events on Kestrel's app, and those are the people who complain.

Why the tail, and not the middle

- ◆ **A long-tailed distribution has no typical value.** The mean sits in a gap where relatively few observations actually are.
- ◆ **Improving the mean can leave the tail untouched**, so the metric moves and the complaints do not.
- ◆ **The tail is a population, not an outlier.** One percent of 892 million events is nine million of them.
- ◆ **And the tail is where compound effects live.** A session touching ten screens meets the ninetieth percentile roughly once.

THE TAKEAWAY

The last point is the one people underrate. Percentiles compound across a journey, so a p90 per screen is not a p90 per session.

Median is the most requested statistic in analytics, and SQL makes it **the most awkward**.

Median, by engine

ENGINE	HOW YOU WRITE IT
Redshift, Snowflake, Oracle, DuckDB	MEDIAN(x). Native, and the shortest form.
PostgreSQL	PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x). No MEDIAN.
SQL Server	The same, plus OVER (). The OVER is compulsory.
BigQuery	PERCENTILE_CONT(x, 0.5). Arguments reversed.
MySQL	Neither. A ranking trick, or a user-defined function.

THE TAKEAWAY

Five engines, four syntaxes, one statistic. No form of this expression runs everywhere, which makes median a real portability problem.

Postgres **forbids** OVER on these functions.
SQL Server **requires** it.

The same median, two engines

POSTGRESQL

OVER is rejected

```
1 SELECT region_key,  
2         PERCENTILE_CONT(0.5)  
3         WITHIN GROUP (ORDER BY basket_value)  
4         AS median_basket  
5 FROM vw_basket  
6 GROUP BY region_key;
```

An aggregate. It collapses rows, needs a GROUP BY, and cannot be written with OVER at all.

SQL SERVER

OVER is mandatory

```
1 SELECT DISTINCT region_key,  
2         PERCENTILE_CONT(0.5)  
3         WITHIN GROUP (ORDER BY basket_value)  
4         OVER (PARTITION BY region_key)  
5         AS median_basket  
6 FROM vw_basket;
```

A window. It keeps every row, so DISTINCT is needed to get back to one row per region.

THE TIEBREAKER

Note the DISTINCT on the right. Because it is a window, every basket comes back carrying its region's median, and the rows have to be collapsed afterwards.

What that means for a rolling median

- ◆ **Snowflake and Redshift allow the window form with a PARTITION BY only.** No ORDER BY inside OVER, and no frame.
- ◆ **So a rolling 90-day median is not expressible as a window function** on any engine in common use. There is no frame to slide.
- ◆ **Join a date spine to itself instead**, or aggregate per window period, and compute an ordinary median per group.
- ◆ **Which is expensive by nature.** Every group needs its own sort, and the groups overlap by design.

THE TAKEAWAY

A running total is one accumulator. A running median is a fresh sort for every row, which is why no engine offers it as a frame.

Quartiles in one call

```
1 SELECT region_key,  
2         PERCENTILE_CONT(ARRAY[0.25, 0.5, 0.75])  
3         WITHIN GROUP (ORDER BY basket_value) AS quartiles  
4     FROM vw_basket  
5     GROUP BY region_key;  
6  
7 -- quartiles -> {14.20, 18.44, 27.90}
```

2 Postgres accepts an array of proportions and returns an array of values, from a single sort.

THE TAKEAWAY

Three percentiles from one sort rather than three. Not portable, and a real saving when the group is large and the sort is the cost.

Two directions, one distribution

ROW IN

CUME_DIST, PERCENT_RANK

- You have a value, want its position
- Returns a proportion
- A window function
- One result per row

PROPORTION IN

PERCENTILE_CONT, DISC

- You have a position, want its value
- Returns a value
- An ordered-set aggregate
- One result per group

THE TIEBREAKER

They are inverses of each other. CUME_DIST of the value PERCENTILE_DISC(p) returned is p, or the next value above it, which is a useful check.

The inverse pair, checked

```
1 WITH d AS (  
2     SELECT basket_value,  
3           CUME_DIST() OVER (ORDER BY basket_value) AS cd  
4     FROM vw_basket WHERE store_key = 41  
5 )  
6 SELECT MIN(basket_value) AS via_cume_dist,  
7       (SELECT PERCENTILE_DISC(0.9) WITHIN GROUP (  
8         ORDER BY basket_value)  
9         FROM vw_basket WHERE store_key = 41) AS via_percentile  
10 FROM d WHERE cd >= 0.9;
```

6 The smallest value whose cumulative share reaches 0.9. That is the definition of PERCENTILE_DISC(0.9).

THE TAKEAWAY

Two routes to one number, and they must agree exactly. If they do not, one of the two is being computed over a different set of rows.

A median of medians **is not a median**. It is not approximately one either.

Why medians cannot be combined

- ◆ **A median throws away everything except a position.** Once you have it, the distribution it came from is gone.
- ◆ **So no amount of per-group medians reconstructs the whole.** Unlike sums and counts, which carry enough to be recombined.
- ◆ **Weighting does not fix it.** A weighted median of medians is still not the median of the underlying values.
- ◆ **Compute it once, from the detail.** That is the only way, and it means percentiles cannot be pre-aggregated by group.

THE TAKEAWAY

This is the practical cost of robustness. The same property that makes a median ignore outliers makes it impossible to combine.

Kestrel's group pack reports a national median basket value. It is built from the existing store-level table, which already holds a median basket per store, by taking the median of those 412 figures.

A new analyst recomputes it from basket detail and gets \$16.80 against the pack's \$19.15. Both figures are reproducible. The store medians range from \$9.40 at small convenience sites to \$34.10 at large superstores.

What is the correct national median?

- ☐ A Neither yet. Weight the store medians by basket count before taking the median of them.
- ☐ B \$16.80. A median of medians is not a median, and it has to be computed once over all baskets.
- ☐ C Take the mean of the 412 store medians instead, which is at least an unbiased summary of them.
- ☐ D The store medians should use PERCENTILE_DISC, so that the national figure is a value that really occurred.

◆ PAUSE AND DECIDE

What is the correct national median?

A Weight the store medians

A weighted median of medians is still not the median. It also needs a weighted median function, which no engine in common use provides.

C Mean of the medians

A different statistic again, with its own weighting problem from module seven. It answers neither question anybody asked.

B Compute once from detail

Medians carry no information that can be recombined. The only correct national median is the one taken over all baskets at once.

D Switch to PERCENTILE_DISC

Changes each store's figure slightly and leaves the combination invalid. The defect is the nesting, not the interpolation.

THE SIGNAL

"\$9.40 to \$34.10." A wide spread of group medians is what makes the error large. On uniform groups the two figures would nearly agree, and the method would still be wrong.

What percentiles cost

- ◆ **Every group needs a full sort of its values.** There is no accumulator and no incremental form, for any percentile.
- ◆ **And they cannot be pre-aggregated by group,** which is the usual escape route for an expensive measure.
- ◆ **Reduce the row count first, at a grain that is still valid.** Basket grain rather than line grain is a 15-fold reduction.
- ◆ **Or accept an approximation,** where the figure is a dashboard tile rather than something anybody reconciles.

THE TAKEAWAY

Sums roll up and medians do not. That single difference is why a percentile over a large table is expensive in a way a total never is.

Approximate quantiles

- ◆ **Most warehouses offer one.** APPROX_PERCENTILE, APPROX_QUANTILES, or a sketch-based function under another name.
- ◆ **They avoid the full sort**, using a summary structure, and are dramatically cheaper on large groups.
- ◆ **The answer moves between runs** on some implementations, which module two says is a property worth knowing about.
- ◆ **Never for a reconciled figure.** An approximate median that disagrees with an exact one is a conversation nobody wins.

THE TAKEAWAY

The same rule as approximate distinct counts in module seven. Fine for a tile that informs a decision, wrong for a number that settles one.

Percentiles and the grain

- ◆ **A percentile computed after a fan-out is meaningless.** Duplicated entities are counted more than once in the ordering.
- ◆ **And it does not fail loudly.** The figure moves toward whichever entities were duplicated, by an amount nobody can estimate.
- ◆ **Fix the grain before asking for the percentile,** not afterwards. There is no correction you can apply later.
- ◆ **Median basket means basket grain.** Computing it over sales lines gives the median line value, at fifteen times the row count.

THE TAKEAWAY

A sum survives a fan-out badly and visibly, because the total is too big. A percentile survives it badly and invisibly.

A store dashboard computes median basket value per store per day directly over fact_sales_line, grouping to basket inside the query. It runs hourly, takes nine minutes, and the requirement is under one.

The measure is published to store managers as guidance and is not reconciled to finance. The fact table holds 2.41 billion lines, and a basket averages roughly fifteen lines.

What is the right change?

- ☐ A Increase the warehouse size, since a percentile is inherently sort-bound and more memory removes the spill.
- ☐ B Precompute a median per store per day nightly, and take the median of those for any longer period.
- ☐ C Materialise a basket-grain table, compute the percentile over that, and use an approximate quantile for the tile.
- ☐ D Replace the median with the mean, which needs no sort and can be computed incrementally.

◆ PAUSE AND DECIDE

What is the right change?

A Bigger warehouse

Reduces the spill and still sorts 2.41 billion rows every hour. It buys time against a table growing by two million rows a day.

C Basket grain, then approximate

A fifteen-fold row reduction at a grain the measure is defined on, and the figure is guidance rather than reconciled.

B Precompute and combine

The daily figures would be right and combining them would not. This is scenario three, proposed as an optimisation.

D Use the mean instead

Cheaper, incremental, and a different statistic. Basket values are right-skewed, which is why the median was chosen.

THE SIGNAL

"Guidance, and not reconciled to finance." That one sentence is what makes an approximation acceptable here and would forbid it in the finance pack.

Choosing, in four questions

01

Direction

Row in and proportion out, or proportion in and value out? That picks the half of the family.

02

Zero

Should the best row read 0 or 1?
PERCENT_RANK or
CUME_DIST.

03

Real

Must the answer be a value that occurred?
PERCENTILE_DISC if yes,
CONT if not.

04

Combining

Will anybody roll this up? If so, store the detail, because a percentile will not roll up.

THE TAKEAWAY

Step four is the one that reaches furthest. It is a modelling decision made while writing a report, and it constrains every report that comes after.

Testing a percentile

- ◆ **Assert it lies between the minimum and the maximum.** Trivial, and it catches a wrong column in the WITHIN GROUP.
- ◆ **Assert the percentiles are monotonic.** P25 at or below P50 at or below P75, which catches a reversed ordering.
- ◆ **Assert the pair agree.** CUME_DIST of the discrete percentile equals the proportion you asked for.
- ◆ **Assert the row count the figure covers,** and publish it, because nulls and small groups both hide there.

THE TAKEAWAY

Four assertions, and none of them needs a second system to compare against. Percentiles are unusually self-checkable, and almost nobody checks them.

The percentile assertion

```
1 SELECT COUNT(*) FILTER (WHERE NOT (p25 <= p50 AND p50 <= p75))
2     AS not_monotonic,
3     COUNT(*) FILTER (WHERE p50 < min_v OR p50 > max_v)
4     AS out_of_range,
5     COUNT(*) FILTER (WHERE n < 30)
6     AS groups_too_small
7 FROM report;
```

5 Not an error, but a flag. A median over fewer than thirty observations should be labelled rather than published bare.

THE TAKEAWAY

The first two must be zero. The third is a judgement, and the number thirty is a convention rather than a rule, which is exactly why it should be written down.

A percentile is a claim about a population
that cannot be rolled up, corrected, or
recombined.

The confusions, settled

SYMPTOM	THE CAUSE
The best performer reads 0.00	PERCENT_RANK starts at zero by construction.
The median is not in the data	PERCENTILE_CONT interpolated between two values.
Two identical rows in different buckets	NTILE divides by count. Use CUME_DIST.
Group and total medians disagree	A median of medians. Not a median at all.
The query is rejected with OVER	PostgreSQL. It has no window form of these functions.
The query is rejected without OVER	SQL Server. It has no aggregate form.

THE TAKEAWAY

The last two rows are the same query, failing on two engines for opposite reasons. There is no way to write it once for both.

Five habits

- ◆ **Use CUME_DIST unless you specifically want a zero-to-one scale.** It removes an entire class of stakeholder confusion.
- ◆ **Ask whether the number must be traceable** before choosing CONT or DISC. Auditors always say yes.
- ◆ **Never nest a percentile.** Not medians of medians, not percentiles of percentiles, not weighted versions of either.
- ◆ **Store detail wherever a percentile will be needed at more than one level,** because it cannot be rolled up.
- ◆ **Write the engine's native form and isolate it,** since there is no median expression that runs on all four dialects.

THE TAKEAWAY

The third habit is the only one that prevents a meaningless number. The other four prevent confusion, arguments and rewrites, which are cheaper.

Nine modules on one clause. **Part 3** is about query structure, and it starts with a piece of folklore.